

Intersect Budget 2026 — Internal Audit

Ballot: Cardano Budget Process 2026

Admin / voting authority: addr1vyk44xqtv3jpk90fuzqfsjzcv259hshuw2qmxq98n3n4chyu9hr

Ballot fingerprint (601): c66f333a5b73775a8938051a3e9779008f9287cc1b9664e096263151

Policy ID: 378b5b28b5effd32f6912c62a7ca5305c24b768ea1fb34e2b9720ec8

Voting window: 2026-05-26T12:00:00Z to 2026-06-12T12:00:00Z (end epoch 636)

Questions: 69

Voters in proof package: 113

Reporting date: 2026-06-16

1. Summary

The ballot's cryptographic evidence chain is intact and has been independently reproduced. The on-chain commitments, the published results, the evidence Merkle root, every per-voter inclusion proof, and the full re-tally of all 69 questions all verify. Both audits agree on this.

Three items require disposition. None of them affects the structural integrity of the ballot, and each is bounded and individually enumerable:

#	Item	Audit signal	Class	Disposition
A	One multisig voter's co-signer signature does not match the signed payload	1 FAIL	Representational signing defect (backend)	Cryptographically failing as recorded; both signers forensically confirmed to have authorized identical content; referred for governance decision
B	All 113 voters signed a ballot identifier that differs from the settled (601) identifier	1 WARN (aggregate)	Settlement-metadata mismatch (middleware)	Benign; uniform; no effect on tally
C	Three voters carry no eligible voting power (one stake credential, two unregistered DReps)	Eligibility layer	Eligibility / source-of-authority	Stake voter excluded; two unregistered DReps assigned zero voting power

The certified, voting-power-weighted result is sound. The adjustments below are limited to (A) one voter pending a governance decision and (C) eligibility exclusions, both of which are quantifiable and do not alter the verified cryptographic chain.

2. Verified properties

The following were confirmed by the internal verifier and independently reproduced by the TypeScript audit:

- On-chain (600) and (601) inline datums decode and are internally consistent.
- The IPFS ballot JSON reconstructs the on-chain (600) Merkle root ([85a76c0d...](#)).
- [results.json](#) hashes to the on-chain (601) results hash ([dc93af95...](#)).
- The proof package root matches the on-chain evidence Merkle root ([c4a9ca80...](#)); 113 voters.
- All 113 per-voter inclusion proofs walk back to the committed evidence root.
- Every committed per-voter evidence file matches its committed vote hash.
- Every vote-history chain is intact, correctly linked, and inside the voting window.
- The (601) UTxO traces directly to a Hydra fan-out with no intervening rebalances.

- An independent recount matches the published per-role counts for all 69 questions.

The integrity, completeness, and tabulation of the ballot are therefore established. The remaining items concern (A) one signature representation, (B) one settlement-metadata label, and (C) voter eligibility, none of which touches the evidence chain or the count.

3. Finding A: Multisig co-signer signature mismatch

- **Voter:** 2293c55fa571fcff89ef1e02de506c3df9dbcf07d601d2f0bb828e5db
- **Credential:** native-script DRep, **all**-of-2 (both keys required): 2d9ac6778912... (member A) and 8685dbde8cc2... (member B).
- **Internal audit signal:** Step 7 FAIL; independent audit: Finding 1.

3.1 Observation

The committed vote (version 2, 69 selections) carries two witnesses. Member A's signature is over the correct signed-payload Merkle root (5263e4b28f1247ca...). Member B's signature is over a different value (b554982a1236e2a9...). Because the credential is an **all**-of-2 script, both required signatures must verify against the same signed message; one does not, so the multisig authorization is not satisfied as recorded.

3.2 Root cause

The backend computes two distinct hashes for a vote: the **signed-payload Merkle root** (`blake2b_256(JSON.stringify(signingPayload))`), the value the voter must sign and the value Hydra verifies), and an internal **evidence voteHash** (`blake2b_256` of the full evidence object, a superset of the signing payload). These values are necessarily different.

The first signer signs the Merkle root returned by the draft endpoint. A co-signer who retrieves the package to sign was served the stored evidence `voteHash` in place of the Merkle root, and signed that. The two co-signers therefore signed two different representations of the same vote. This was not caught at submission time because neither the backend signature endpoint nor the middleware verified the constituent signatures against the Merkle root before accepting and settling the package.

3.3 Forensic determination

The discrepancy is representational, not substantive. We have confirmed, by reconstruction that both members authorized the **identical** ballot content:

- `merkleRoot(signingPayload)` over the 69 selections is 5263e4b28f1247ca..., the value member A signed.
- `voteHash(evidence)` over the **same** 69 selections is b554982a1236e2a9..., the value member B signed.

Both values regenerate exactly from the same signed payload. Both signatures are valid Ed25519 signatures by the two correct member keys named in the script. The event is therefore a genuine,

fully authorized vote affected by a signing-target defect. It is not a forgery, not a content disagreement between the signers, and not a single-signer bypass: both keyholders independently signed, and both approved the same selections.

Appendix A is a complete, self-contained recipe (source data, both hash constructions, and the signature checks) that allows any third party to reproduce every value in this section independently.

3.4 Disposition

- **Cryptographic (trustless) layer:** the vote does not satisfy the **all**-of-2 verification as recorded, and the verifier correctly reports it as failing. It is **not** silently reclassified as passing.
- **Forensic layer:** both required signers are confirmed to have authorized identical content, and this determination is documented and reproducible.
- **Decision required:** whether to count this voter is a governance decision for the voting authority, not a cryptographic one. Recommendation: count the vote on the basis of the reproducible dual-signature evidence above, recording this response as justification. The voter is held as *cryptographically failing, forensically authorized*.
- **Scope:** one voter of 113. The selections are known, and the effect of including or excluding this single ballot on the weighted result is fully determinable.

3.5 Note on the independent verifier's treatment

The independent audit reported this voter as "accepted as valid" after updating its verifier to classify script DRep credentials separately and skip the direct key-hash match. That relaxation is correct in one respect, since a script credential is not the Ed25519 key hash and a direct key-hash comparison is therefore the wrong test, but it went one step further than intended: by skipping straight to acceptance it stopped verifying the individual witness signatures and the script's M-of-N threshold. The correct procedure for a script credential is to (1) confirm the native-script hash equals the credential, (2) verify each witness signature against the Merkle root, and (3) require the set of *verified* signers to satisfy the script rule. Under that procedure the **any**-of-3 script voter ([227d74b5...](#)) genuinely passes on its one valid signature, and this **all**-of-2 voter genuinely fails. We recommend the independent verifier adopt the per-signature/threshold check, so the two tools agree.

4. Finding B: Uniform ballot-identifier mismatch

Internal audit signal: Step 7 aggregate WARN (all 113 voters).

All 113 voters signed ballot identifier `6a1512d782978c99456fe6de`, while the settled (601) datum records the 28-byte fingerprint `c66f333a5b73775a8938051a3e9779008f9287cc1b9664e096263151`.

This is a settlement-metadata discrepancy. The Hydra head re-seeded its identity during the ballot's lifecycle (a restart/rehydration) and, at settlement, fell back to the fingerprint default instead of the custom identifier the voters had signed against. The condition is **uniform**: every voter agrees on a single signed identifier. A vote-replay scenario would instead present divergent identifiers, which is not the case here. Every other binding (policy ID, voter token, signed-payload Merkle root, committed evidence root, and vote content) ties each ballot unambiguously to this ballot.

Disposition: benign, with no effect on tabulation or integrity. Reported as a warning rather than a failure. The middleware defect (preserving the cached ballot identifier across a re-seed) is tracked under corrective actions.

5. Finding C: Voter eligibility

The cryptographic audit establishes that a signature is genuine and that a vote is correctly committed. Eligibility, meaning whether the signer is a member of the authorized voter set for this ballot, is a separate gate. Three voters require disposition.

5.1 One stake-credential voter: excluded

- **Voter:** `e02d7e2c16a003a17a4c4f89673a7f1329f381ca9c9ea08ade2e4ce0af`
- **Type:** stake credential.
- **Reported voting power:** 31,693,327,692 lovelace (approximately 31,693 ADA).

This ballot's authorized voter set is DRep credentials. A stake credential is not a member of that set. A middleware eligibility-gate misconfiguration permitted this stake credential to pass validation and cast a vote. The vote is cryptographically valid, but cryptographic validity does not confer eligibility.

Disposition: **excluded** from the certified tally as an ineligible voter. The independent audit resolved this entry by adding a corresponding `epoch_stake` snapshot row; that confirms the credential exists and carries stake on-chain, but on-chain existence is not eligibility for a DRep ballot. The voter and its associated voting power are removed from the certified result.

5.2 Two DRep voters never registered on chain: zero voting power

Voter token	DRep ID	Credential
220060a4f5b30c6f0b03a3e6875f0fa26e4f8ec475869ec7ada0282e88	drep1ygpl05fplelgu35uc7ljx8tc0j3ld9glas99qvfrgtgm8rg4ujzg7	03f7d121fe7e8e469cc7bf231d787ca3f6951fec0a50312342d1b38d
22b32ee7b325d63c41a65a976cc065b6ad0e7920b02d65f78321d85082	drep1yf5kddtr03tp450dkkxymk3nws3qwnux55kwztkj2v3z5eshqugjs	6966b5637c561ad1edb58c4dda337422074f86a52ce12ed253222a66

Both voters present valid signed evidence, valid DRep key credentials, and intact in-window vote-history chains. Neither appears in the epoch-636 [drep_distr](#) snapshot (888 DRep entries), a current Blockfrost lookup returns 404 for both DRep IDs, and manual db-sync checks against [drep_hash](#) / [drep_distr](#) return no rows. We confirm that these two DRep credentials were never registered on chain.

Design note: registration is intentionally not enforced at vote time. The voting system does not gate vote acceptance on on-chain DRep registration state. This is deliberate. A participant may generate a credential key pair, cast a signed vote, and register that credential on chain afterward, a sequence the Hydra head cannot anticipate when a vote is received, because a future registration is not observable at that moment. The system therefore accepts a cryptographically valid vote and resolves eligibility and voting power at certification, against the authoritative on-chain snapshot. The signature proves authorship, and the snapshot determines weight. This avoids both false rejection of a voter who registers slightly later and any dependence on registration state the head cannot see.

Disposition: Because these two credentials were never registered on chain, they carry no registered voting power. They are assigned **zero voting power** and are **excluded from the weighted result**. Their signed votes are genuine and remain in the evidence record but contribute nothing to the certified tally. This follows directly from the eligibility model above: an unregistered credential has zero weight by construction, independent of whether registration never occurred (as here) or merely post-dates the snapshot.

6. Impact on the certified result

- **Cryptographic integrity:** unaffected. Root reconstruction, result hash, evidence root, all inclusion proofs, and the independent re-tally verified.
- **Tabulation:** the published per-role counts matched independent re-derivation for all 69 questions. Recorded participation: 113 voters (112 DRep, 1 stake); 110 active and 3 abstain-only.
- **Adjustments to the certified, voting-power-weighted tally:**
 1. Remove the one stake-credential voter (Section 5.1) as ineligible.
 2. Apply the governance decision for the one multisig voter (Section 3.4).

3. Assign zero voting power to the two unregistered DRep voters (Section 5.2); they carry no registered weight and do not affect the weighted result.
- All adjustments are enumerable and do not alter the structural soundness of the ballot. Four of 113 ballots are in question: three eligibility determinations (one excluded, two zero-weight) and one signing-representation item. The two unregistered DReps have no weighted effect by construction.

7. Corrective actions

Backend (signing flow):

- Serve cosigners the signed-payload Merkle root as the signing target, never the internal evidence `voteHash`. Recompute the Merkle root from the stored signing payload on every package retrieval.
- Verify each incoming witness signature against the Merkle root (honoring the COSE `hashed` header) before storing it, and reject otherwise.
- Count only verified signatures toward the script threshold and re-assert a "valid package" invariant (every witness verifies and the script is satisfied by verified signers) before submitting to the middleware. The backend must not submit an unverifiable package.
- Add unit, integration, regression, and contract tests enforcing the above, including a regression case reproducing this ballot's multisig voter.

Middleware (Hydra):

- Verify script-witness signatures against the Merkle root and enforce the native-script M-of-N rule, rather than checking key membership alone.
- Preserve the cached ballot identifier across head re-seed and rehydration so the settled (601) identifier matches the identifier voters signed.
- Honor the COSE `hashed` header in signature verification.

Eligibility gate:

- Enforce credential-type restriction per ballot: a DRep-scoped ballot must reject non-DRep credential types (including stake credentials) at validation time. This is a credential-type check and does not depend on the registration state.
- Continue to resolve eligibility and voting power at certification against the authoritative on-chain snapshot, not at vote-acceptance time, so that an unregistered credential is correctly assigned zero weight, consistent with the design rationale in Section 5.2.

Audit tooling:

- The internal verifier now performs full script-credential verification (script hash match, per-witness signature verification, M-of-N threshold) and reports the uniform ballot-identifier mismatch as an aggregate warning. We recommend the independent verifier adopt the same per-signature/threshold check for script credentials (Section 3.5).

8. Conclusion

The cryptographic evidence chain for Cardano Budget Process 2026 is sound and has been reproduced by two independent verifiers. The single cryptographic failure is explained, root-caused, and forensically resolved to a representational signing-target defect affecting one voter whose underlying selections are confirmed and reproducible. The uniform ballot-identifier warning is a benign settlement-metadata condition. The eligibility items are bounded: one stake credential is excluded as outside the authorized voter set following a middleware misconfiguration, and two DRep credentials that were never registered on chain are assigned zero voting power in accordance with the system's eligibility model. Corrective actions are specified across the backend, middleware, eligibility gate, and audit tooling, with testing mandated so that only verifiable, eligible packages reach settlement in future ballots.

Appendix A: Reproduction of the multisig voter (Finding A)

This appendix lets any third party independently reproduce every value in Section 3 from public data: the two payload hashes, the two member signatures, and the conclusion that both required signers authorized identical content. All hashes are blake2b, and ballot and credential identifiers are as listed at the head of this document.

A.1 Source data

1. From the on-chain (601) datum, read the evidence CID:
`QmRmJzmUzT4jKKNCkUeRsEPQm7eQ9Ti3f9JKEvRCN8J4Wg`.
2. Fetch the committed evidence file for this voter from IPFS:
`<evidenceCID>/vote-2293c55fa571fcff89ef1e02de506c3df9dbcf07d601d2f0bb828e5db-v2.json` (v2 is the committed version; per Step 6 it hashes to the committed vote hash).
The file provides `ekklesia.signedPayload` (ballotId, nonce, the 69 selections),
`ekklesia.witnesses` (the two COSE_Sign1 objects), `voterId`, `credentialHrp`, `nonce`, and `nativeScript`.

Voter and credentials:

- `voterId`: `drep1ywuk39acvmkzda9e8kfmy7fwf946jd5uaa4l696atlas66s68f0fl`
- `nativeScript`: `all-of-2`, requiring `sig`
`2d9ac6778912d070941ab8b996c877bf28b6e6490f6c40ebb3dcbea0` AND `sig`
`8685dbde8cc2205f5bf601fa4b3333bff5bb1ad5490aa9df5c73d9f0`
- `signed ballotId`: `6a1512d782978c99456fe6de`, `nonce`: `2`, `selections`: 69

A.2 Hash 1: the signed-payload Merkle root (what member A signed)

None

```
merkleRoot = blake2b_256_hex( utf8( JSON.stringify(signedPayload) ) )
```

Where `signedPayload` = `{ "ballotId", "nonce", "votes" }` in that key order (equivalently, minified JSON with no whitespace). Expected:

None

```
merkleRoot = 5263e4b28f1247ca69ff1336ef177ee9722b523fe763b07213befb89129ac757
```

A.3 Hash 2: the backend evidence voteHash (what member B signed)

Member B's wallet was served the backend's internal evidence `voteHash` as the signing target (the defect in Section 3.2). Reconstruct that evidence object from the same underlying data using the fixed backend schema:

None

```
{
  "specVersion": "ekklesia/1.0",
  "surveyTxId": "6a1512d782978c99456fe6de",    // equals ballotId
  "responderRole": "drep",                      // derived from credential HRP
  "answers": <the 69 selections, verbatim>,
  "ekklesia": {
    "voterId": "drep1ywuk39acvmkzda9e8kfmy7fwf946jd5uaa4l696atlas66s68f0f1",
    "credentialHrp": "drep",
    "nonce": 2,
    "signedPayload": <the same signedPayload as A.2>,
    "witnesses": [],
    "merkleProof": { "root": "", "steps": [] }
  }
}
```

Hash it with canonical JSON (RFC-8785-style: object keys sorted recursively, arrays in order, primitives via standard JSON encoding, no insignificant whitespace):

None

```
voteHash = blake2b_256_hex( utf8( canonicalJSON(evidence) ) )
           = b554982a1236e2a98ddca1953f2b586baa3721e8f17d77f75b5ca8b97e58596f
```

A.4 Signature verification

For each of the two COSE_Sign1 witnesses in `ekklesia.witnesses`:

1. Parse `coseSign1Hex` as the CBOR array `[protected, unprotected, payload, sig]` and `coseKeyHex` as the COSE key; the Ed25519 public key is at label `-2`.
2. Confirm the member key hash: `blake2b_224_hex(pubKey)` equals a `sig` keyHash in the native script.
3. Confirm the signed message: the `payload` (ASCII) equals the relevant hash above (the `hashed` header is `false`).
4. Verify the Ed25519 signature over the COSE `Sig_structure ["Signature1", protected, b"", payload]` (CBOR-encoded).

Expected result:

Witness	pubKey	member keyHash = blake2b_224(pu bKey)	Ed25519 valid	signed message
A	a0810fb2e6571f6 936c1933fccac8 0da16e6b9609d1 b8b469a17d1f9b bc1b026	2d9ac6778912d0 70941ab8b996c 877bf28b6e649 0f6c40ebb3dcb ea0	yes	merkleRoot (A.2)
B	56fb22f8705c69 e5662b63480a9 554c7dfdfb8723 7c9ff46b613d6b 521ee02e2	8685dbde8cc22 05f5bf601fa4b33 33bff5bb1ad549 0aa9df5c73d9f0	yes	voteHash (A.3)

A.5 Conclusion of the reproduction

Both signatures are valid Ed25519 signatures by the two key hashes the `all -of-2` script requires. `merkleRoot` (A.2) and `voteHash` (A.3) are both derived from the **same** ballotId, nonce, and 69 selections. Member A signed the Merkle root, and member B signed the evidence voteHash of the identical content. This reproduces, from public data alone, that both required signers authorized the same vote, and that the only discrepancy is which hash each was asked to sign. It establishes the finding without reliance on any backend record.